C Arrays, Pointers and Functions

Indexed data, memory addresses, pointer operations, function interfaces, and embedded engineering practice

Embedded Bare-Metal Diploma

DAY 2

6 Hours

```
int readings[5] = {42, 55, 61, 58, 64};
int *ptr = readings;
float average(const int *data, int
size);
```

Prepared by Mahmoud Morsy

NTI | National Telecommunication Institute

Build from stored data to reusable processing functions

Session roadmap

00:00	01:20	02:00	03:20	04:35
01 Arrays, indexing and traversal	**02** Strings as character arrays	**03** Addresses and pointers	**04** Functions and parameter passing	**05** Integrated practice and assessment

Teaching rhythm: explain → demonstrate → 5–10 minute task → review

By the end, students can connect arrays, pointers and functions

Learning outcomes

- Declare, initialise and traverse one-dimensional arrays.
- Protect array boundaries and calculate basic statistics.
- Explain a string as a null-terminated character array.
- Use & to obtain an address and * to access its value.
- Explain the relationship between arrays and pointers.
- Pass values, addresses and arrays to functions safely.

TODAY'S OUTPUT

8

working programs and completed quizzes

Grouped data turns repeated variables into scalable programs

Why arrays matter

01

One name, many values

Store repeated sensor samples under one array identifier.

02

Predictable traversal

A loop processes every element using the same operation.

03

Reusable interfaces

Functions can receive a buffer and its element count.

Bare-metal mindset: fixed memory is limited—know the capacity, validate every index, and pass the size with the array.

An array stores equal-type elements under one name

Array declaration

```
int readings[5] = {42, 55, 61, 58, 64};

printf("%d\n", readings[0]);
readings[2] = 70;
```

READ IT TOP TO BOTTOM

- **int is the element type.**
- **readings is the array name.**
- **[5] reserves five elements.**
- **Initialiser values fill indices 0–4.**
- **readings[2] accesses the third element.**

The first valid index is zero

Indices map directly to consecutive array elements

Array memory model

VALID: 0–4

Five int elements occupy consecutive locations; index 5 is outside the array and must never be accessed.

Choose the declaration pattern from the data requirement

Array declaration patterns

Requirement	Declaration	Meaning
Five readings	int data[5];	Uninitialised elements
Known values	int data[] = {2,4,6};	Length inferred as 3
All zeros	int data[5] = {0};	Every element becomes 0
Decimal samples	float v[4];	Four float elements
Text buffer	char cmd[8];	Capacity: eight chars

Important: a local array length is fixed after declaration; initialise unused elements deliberately and keep the element count available to every loop and function.

Exercise: choose the correct array declaration

7-minute practice

1. Store five integer temperatures

2. Store four decimal voltages

3. Create six integers initially equal to zero

4. Store MOTOR as a C string

Work individually • 7 minutes

Answer: type, capacity and initial values must agree

Instructor review

```
int temperatures[5];
float voltages[4];
int counters[6] = {0};
char motor[] = "MOTOR";
```

ASK WHY

Why does MOTOR need six char positions?

Which declarations are fully initialised?

Indexing reads or changes exactly one array element

Indexing and bounds

```
int data[4] = {10, 20, 30, 40};

int x = data[1];
data[3] = 45;
// data[4] is invalid
```

INDEX RULES

Expression	Meaning
data[0]	First element
data[3]	Fourth element
data[n-1]	Last valid element
data[n]	Outside the array

Valid indices run from 0 to length − 1. An out-of-range access produces undefined behaviour.

Most array tasks use the same traversal pattern

Array operations

Operation	Typical code	Purpose
Read	value = data[i];	Use one element
Write	data[i] = value;	Change one element
Traverse	for (i = 0; i < n; i++)	Visit all elements
Accumulate	sum += data[i];	Combine values
Compare	if (data[i] > max)	Find an extreme

Common error: using i <= SIZE visits one invalid element; use i < SIZE for an array containing SIZE elements.

Exercise: calculate the average of five readings

8-minute practice

avg = sum ÷ 5

- Store five integer readings.
- Use a loop to accumulate the sum.
- Convert before division to keep the decimal result.

Example: 40, 50, 60, 70, 80 → average = 60.0

Answer: one loop accumulates every array element

Instructor review

```
int readings[5] = {40, 50, 60, 70, 80};
int sum = 0;

for (int i = 0; i < 5; i++)
{
    sum += readings[i];
}

float average = sum / 5.0f;
printf("%.1f\n", average);
```

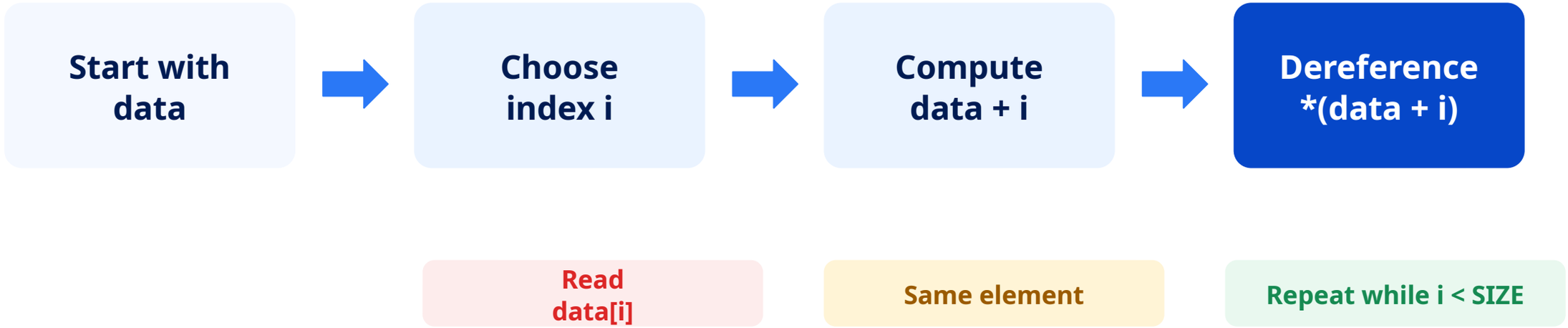
INPUT

PROCESS

OUTPUT

Indexing is translated into an address and a value

From arrays to pointers



Array indexing hides an address calculation; pointers make that calculation visible.

The loop index selects the current array element

Traversal example

```
int samples[5] = {12, 18, 15, 21, 17};

for (int i = 0; i < 5; i++)
{
    printf("%d ", samples[i]);
}
```

TRACE THREE INDICES

0	12
1	18
2	15

Exercise: find the minimum and maximum samples

8-minute practice

Index	Sample
0	12
1–3	18, 15, 21
4	17

REQUIREMENTS

Set min and max from samples[0].

Compare indices 1–4.

Print both results.

Work individually • 8 minutes

Answer: initialise from real data, then compare the remainder

Instructor review

```
int min = samples[0];
int max = samples[0];

for (int i = 1; i < 5; i++)
{
    if (samples[i] < min) min = samples[i];
    if (samples[i] > max) max = samples[i];
}
```

BOUNDARY CHECK

For 12, 18, 15, 21, 17:

minimum = 12

maximum = 21

Strings are char arrays terminated by \0

Strings as arrays

```
char command[8] = "START";

printf("%c\n", command[0]);
printf("%s\n", command);
```

REMEMBER

- START contains five visible characters plus \0.
- The buffer capacity must include the terminator.
- String functions stop when they reach \0.

This short bridge prepares students to treat text as stored array data.

A pointer stores the address of another object

Pointer concept

Object

speed

Stores 1200

Address

&speed

Identifies its location

Pointer access

ptr / *ptr

Stores address / accesses value

A pointer value is an address—not the measured data itself.

A pointer creates another route to the same memory

Pointer memory model

```
int s = 1200;
int *p = &s;
```

Follow the stored address, then dereference it.

p stores &s; dereferencing *p accesses the same value stored in s.

Exercise: trace a pointer update

7-minute practice

```
int value = 10;
int *p = &value;
*p = 25;
```

- Predict the final value of value.
- State what is stored inside p.
- State what *p produces.

Then add 5 to value through p.

Work individually • 7 minutes

Answer: dereferencing writes to the pointed object

Instructor review

```
int value = 10;
int *p = &value;

*p = 25;
*p = *p + 5;

printf("%d\n", value);
```

TRACE

10 → 25 → 30

Final value = 30

In most expressions, an array name points to its first element


```
int data[4] = {10, 20, 30, 40};
int *p = data;

printf("%d\n", *p);
printf("%d\n", *(p + 2));
```

First value

***data**

Indexed value

data[i]

Pointer form

***(data + i)**

Element address

data + i

Pointer arithmetic advances by elements and must remain within the same array.

Exercise: traverse an array using a pointer

8-minute practice

```
for (int *p = data; ... )
```

- Start p at the first element.
- Continue while p is before data + 4.
- Print *p.
- Increment p once per iteration.

Use data = {10, 20, 30, 40}.

Maximum time • 8 minutes

Answer: the pointer advances one element each iteration

Instructor review

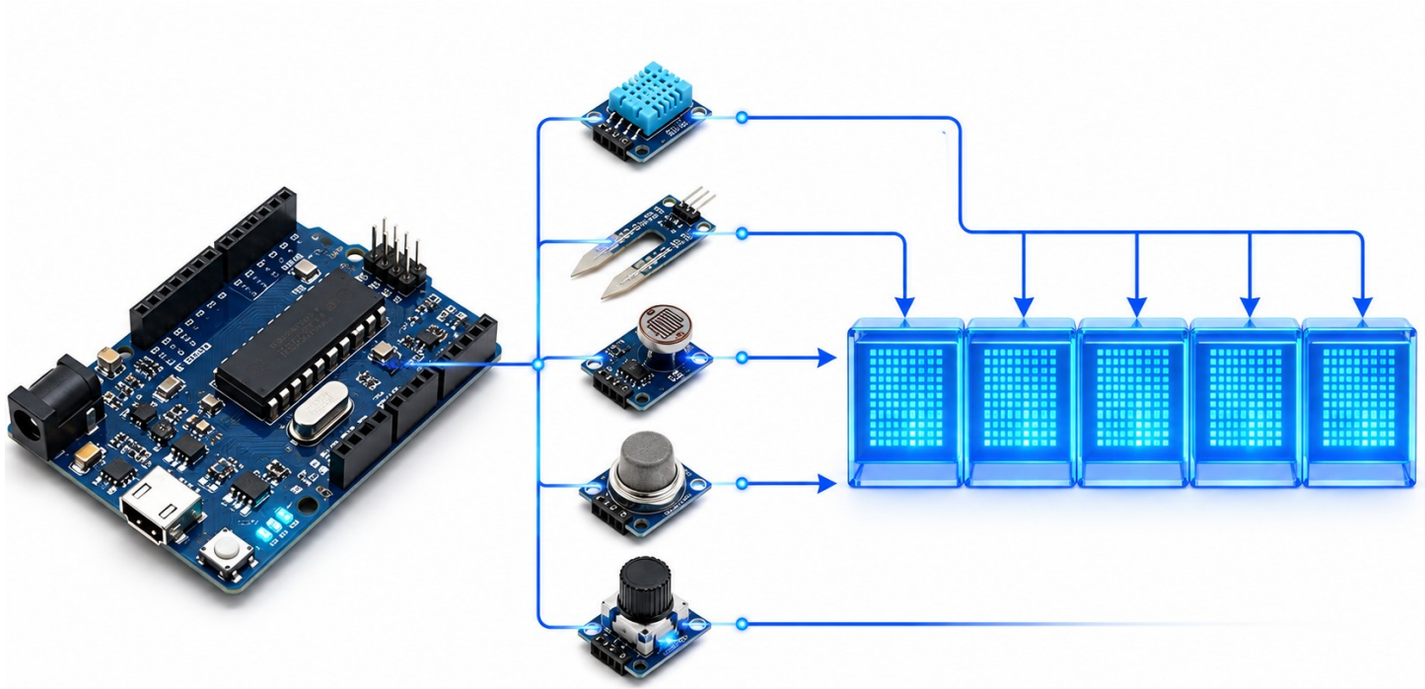
```
int data[4] = {10, 20, 30, 40};

for (int *p = data; p < data + 4; p++)
{
    printf("%d ", *p);
}
```

OUTPUT

10 20 30 40

A sensor buffer becomes reusable through a function

Function design



Sensor Buffer → average()

Interface part	Purpose
const int *data	Input buffer
int size	Element count
float return	Calculated average

Concept demonstration

A function packages processing behind a clear interface

Function anatomy

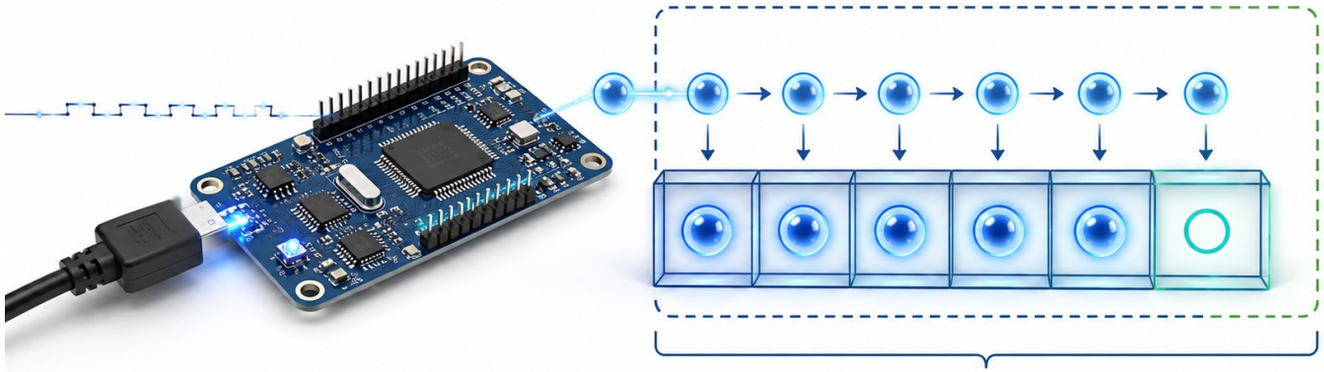
```
float average(const int *data, int size)
{
    int sum = 0;
    for (int i = 0; i < size; i++)
        sum += data[i];

    return (float)sum / size;
}
```

```
int readings[5] = {42, 55, 61, 58, 64};
float result = average(readings, 5);

printf("Average: %.1f\n", result);
```

ANATOMY **return type • function name • parameters • body; pass the array address together with its element count**

Challenge: predict value passing and address passing

HackerRank-style application



INPUT

```
motorSpeed = 50
```

CALLS

setCopy(motorSpeed)

setAddress(&motorSpeed)

Maximum time • 10 minutes

Value parameters copy; pointer parameters reach the original

Application review

```
void setCopy(int speed)
{
   speed = 100;
}
```

```
void setAddress(int *speed)
{
   *speed = 100;
}
setCopy(motorSpeed);      // 50
setAddress(&motorSpeed);  // 100
```

OUTPUT: final motorSpeed = 100

Final quiz — arrays and pointers

10-minute assessment • Part 1

1 What is the last valid index of int data[5]?

2 What does &value produce?

3 What does *ptr access?

4 Complete: data[i] is equivalent to _____

Final quiz — functions

10-minute assessment • Part 2

5 Name the four main parts of a function definition.

6 Why must an array function receive its element count?

7 How can a function modify a caller’s variable?

8 What does const int *data prevent through data?

Quiz answers

Instructor review

1 4	**2** the address of value
3 the object at ptr’s address	**4** *(data + i)
5 return type, name, parameters and body	**6** the array parameter does not carry its count
7 pass its address and dereference it	**8** modifying the array elements

Revise missed concepts before Day 3.

Arrays, pointers and functions form one reusable data path

Session synthesis

01

Arrays

organise repeated samples

02

Addresses

identify stored objects

03

Pointers

access data indirectly

04

Functions

package operations behind interfaces

Next: structures, bitwise operations, registers and modular driver organisation

Teaching references

Sources

CORE COURSE MATERIAL

GNU C Language Manual — arrays, pointers and function parameters
AVR-LibC — fixed-width types and string functions

PRACTICE STRUCTURE

W3Schools — arrays, pointers and functions examples
HackerRank — input–process–output C practice format

Links are available in the PowerPoint speaker notes.

All applications, quizzes, code solutions and explanatory layouts in this deck are original course materials. References guided the technical explanations and practice structure; no proprietary exercise was copied.